

北京邮电大学 本科毕业设计（论文）中期进度报告

Project Mid-term Progress Report

学院 School	International School	专业 Programme	Intelligent Science and Technology		
姓 Family name	赵 Zhao	名 First Name	哲昀 Zheyun		
BUPT 学号 BUPT number	2022213670	QM 学号 QM number	221170559	班级 Class	2022215124
论文题目 Project Title	Network Resource Management Based on Language Model Multi-Agent Systems				
是否完成任务书中所定的中期目标? Targets met (as set in the Specification)? YES					
已完成工作 Finished work: 1. Research Survey and Problem Definition 5G network slicing technology enables the creation of multiple logical networks over a shared physical infrastructure, with each slice serving differentiated service requirements [13]. Enhanced Mobile Broadband (eMBB) pursues high throughput, Ultra-Reliable Low-Latency Communication (URLLC) is designed for extremely low-latency and high-reliability scenarios, typically characterised by millisecond-level end-to-end latency and stringent reliability targets, and massive Machine-Type Communication (mMTC) must support concurrent access from massive numbers of devices. When these three slice types share limited spectrum resources, their Quality of Service (QoS) requirements become inherently competitive. Heavy bandwidth consumption by eMBB users streaming high-definition video may directly threaten latency guarantees for URLLC users, while periodic mass reporting by mMTC devices can squeeze access resources available to other slices [13]. Traditional network resource management methods, including rule-based heuristic allocation and Deep Reinforcement Learning (DRL)-based optimisation, encounter significant challenges [14, 15]. Although such approaches can encode constraints and priorities explicitly, they generally struggle to integrate 3rd Generation Partnership Project (3GPP) Service Level Agreement (SLA) constraints, operator-defined business priority policies, and dynamically evolving user behaviour patterns into a unified and adaptive decision-making framework. The emergent capabilities of Large Language Models (LLMs) offer a new approach to this problem [17]. Since 2023, researchers have begun exploring LLMs as intelligent decision engines for network management. However, existing work has revealed two core gaps that form the starting point of this research. Research Gap 1: Lack of an end-to-end multi-agent LLM framework. The WirelessAgent framework, implements a complete cognitive architecture consisting of perception, memory, planning, and action, and demonstrates superior performance in network slicing scenarios, reporting bandwidth utilisation 44.4% higher than pure prompting methods under its experimental setting [1]. However, it remains fundamentally a single-agent design in which one LLM is responsible for resource decisions across all slices. Such a centralised structure lacks specialised analytical roles and structured negotiation mechanisms when inter-slice demand conflicts arise. CommLLM proposes a three-component multi-agent architecture, with validation primarily focused on semantic communication rather than network slice resource allocation [3]. Consequently, a complete framework dedicated to network slice resource management that supports explicit multi-agent collaboration and can be validated in an end-to-end slicing environment remains underexplored and warrants further investigation.					

Research Gap 2: Lack of structured multi-agent negotiation and conflict resolution mechanisms. Network slice resource allocation can be formulated as a constrained multi-party optimisation problem in which total bandwidth is fixed and slices compete for limited resources [19]. This setting differs substantially from the predominantly cooperative task structures addressed by general-purpose multi-agent LLM frameworks such as MetaGPT [9] and ChatDev [12]. In competitive slicing scenarios, agents must reconcile conflicting objectives under hard resource constraints and service-level requirements. Systematic negotiation protocols and conflict resolution mechanisms tailored to such competitive environments remain underexplored in the network management literature.

2. Research Objective and Contributions

Research Objective: In a 5G network slicing simulation environment, design and implement an LLM multi-agent negotiation-based resource allocation framework (two-tier scheduling with low-frequency LLM multi-agent policy decisions + high-frequency rule execution), compare it against rule-based and DRL baselines, and validate the improvement of the multi-agent negotiation mechanism in SLA satisfaction rate and resource utilisation.

Core Contributions:

1. **A self-built reproducible 5G network slicing resource management simulation environment** (based on Gymnasium) [20], with clearly defined state space, action space, reward function, and SLA evaluation logic, with all parameters and interfaces made publicly available.
2. **A two-tier scheduling LLM multi-agent decision framework:** the upper tier features multiple specialised LLM agents (slice agents + coordinator agent) collaborating on resource allocation decisions through a negotiation protocol; the lower tier uses a rule executor operating at millisecond granularity. Framework effectiveness is validated through 3 core comparative experiments.

Research Questions:

RQ1: Can a multi-agent LLM framework outperform traditional rule-based strategies and DRL methods in network slice resource management? Corresponding objective: Quantify the performance differences of the multi-agent LLM framework relative to rule-based and DRL baselines in SLA satisfaction rate, resource utilisation, and dynamic load adaptability through comparative experiments.

RQ2: Does the negotiation mechanism between agents provide structural advantages? Corresponding objective: Through a "with negotiation vs. without negotiation (centralised dictatorial allocation)" comparison, validate the value contribution of the negotiation protocol itself, rather than merely the result of "more LLM calls providing more computation."

3. Literature Review

3.1 Diverging Technical Routes for LLM Agent Frameworks

Between 2023 and 2025, LLM agent research for wireless networks has developed along three technical routes, each making different trade-offs in addressing domain adaptation.

The prompting and agent workflow route is represented by WirelessAgent [1] and WirelessLLM [2]. WirelessAgent decomposes network management tasks into a sequential workflow of intent understanding → slice allocation → bandwidth allocation → load balancing, with each stage completed by a LangGraph node managing global state transitions. The core contribution is demonstrating that "cognitive architecture + tool calling" can approach rule-optimal performance without training (only 4.3% lower). WirelessLLM proposes a more complete methodological

roadmap: rapid domain knowledge injection through prompt engineering and RAG (knowledge alignment), fusion with specialised models (knowledge fusion), and model evolution through continual learning [2].

The parameter fine-tuning route is represented by NetLLM [4], which reformulates network optimisation as sequential decision problems. Based on Llama2-7B and LoRA, it outperforms DRL baselines on viewport prediction, adaptive bitrate selection, and cluster job scheduling. However, this route requires large amounts of labelled data, limiting its applicability in data-scarce network slicing scenarios.

The cloud-edge collaboration route was pioneered by NetGPT [5], proposing edge deployment of lightweight LLMs for latency-sensitive inference and cloud deployment of large models for global optimisation.

3.2 Multi-Agent Collaboration: From General Paradigms to Network Adaptation

General-purpose multi-agent LLM research has formed relatively mature methodologies. MetaGPT encodes Standard Operating Procedures (SOPs) as prompt sequences [9], with agents transmitting structured intermediate artefacts rather than free text, significantly reducing information loss. AutoGen [10] provides a flexible conversational programming paradigm. CAMEL [11] reveals the possibility of emergent cooperative behaviour in LLM multi-agent systems.

However, adapting these paradigms to network resource management faces three specific challenges: first, resource allocation is **competitive constrained optimisation** rather than purely cooperative; second, decisions have **strong real-time constraints**; third, decision outcomes have **clear quantitative evaluation criteria** (SLA satisfaction rate, throughput, latency, etc.).

Multi-agent attempts in the networking domain are still in early stages. CommLLM's three-component architecture provides a conceptual framework but lacks implementation details. The dual-loop edge-terminal collaboration by Qu et al.[6] is the most architecturally informative, but coordination remains primarily centralised. ORAN-GUIDE [8] employs an "LLM provides semantic understanding, RL handles online optimisation" division of labour, offering insights for resolving the LLM decision latency vs. network real-time contradiction.

3.3 Positioning of This Research

As summarised in Table 1, compared with WirelessAgent [1] (single-agent slicing decision) and existing multi-agent concepts without end-to-end slicing validation, this research aims to develop an end-to-end network slice resource allocation framework and proposes an explicit proposal–evaluation–negotiation protocol under two-tier scheduling.

Table 1. Comparison of related frameworks and positioning of this research.

Feature	WirelessAgent (2025)	CommLLM (2024)	Qu et al. (2025) [6]	This Research
Agent count	Single agent	Multi-agent (concept)	Dual-loop multi-agent	Multi-agent
Validation scenario	Network slicing	Semantic comm.	Edge-terminal collab.	Network slice resource allocation
Negotiation mechanism	None	Unspecified	Centralised coord.	Proposal-evaluation-negotiation protocol
Real-time handling	No explicit constraint	Not discussed	Inner-loop re-planning	Two-tier scheduling (LLM low-freq + rule high-freq)
Simulation validation	Custom env.	Concept level	Partial validation	Gymnasium standardised env.

3.4 Reproducibility Status and Open-Source Resources

Through verification of repository accessibility and runnable entry points, the open-source resources used as reproducible references in this work are listed in **Table 2**.

Table 2. Verified open-source resources and reproducibility status.

Resource	Source	Core Function
WirelessAgent_R1	github.com/jwentong/WirelessAgent_R1	LangGraph agent + network slicing KB
network-slicing gym [20]	github.com/jjalcaraz-upct/network-slicing	Gymnasium-based RAN slicing RL env.
CommLLM	github.com/jiangfeibo/CommLLM	Multi-agent LLM framework with retrieval, cooperative planning, and evaluation loop
NetLLM	github.com/duowuymys/NetLLM	Adapts LLMs to networking tasks via low-rank fine-tuning and specialised task heads
LangGraph	github.com/langchain-ai/langgraph	State-machine orchestration library for long-chain, multi-agent LLM workflows

Key findings: Several existing academic frameworks rely on closed-source APIs such as GPT-4 and do not release complete experimental code, while WirelessAgent is among the few works that provide a fully executable and reproducible codebase. Based on recently published repositories and engineering implementations, state-graph-based orchestration frameworks such as LangGraph are frequently adopted in related studies, as they facilitate controllable multi-step reasoning and structured tool invocation. To the best of our knowledge, publicly available datasets specifically designed for network slicing, with explicit SLA annotations and detailed workload descriptions, remain scarce. Consequently, most existing studies rely on synthetic traffic models or generate workload sequences directly within simulation environments.

4. System Architecture Design

Network Resource Management Based on Language Model Multi-Agent Systems (5G Network Slicing)

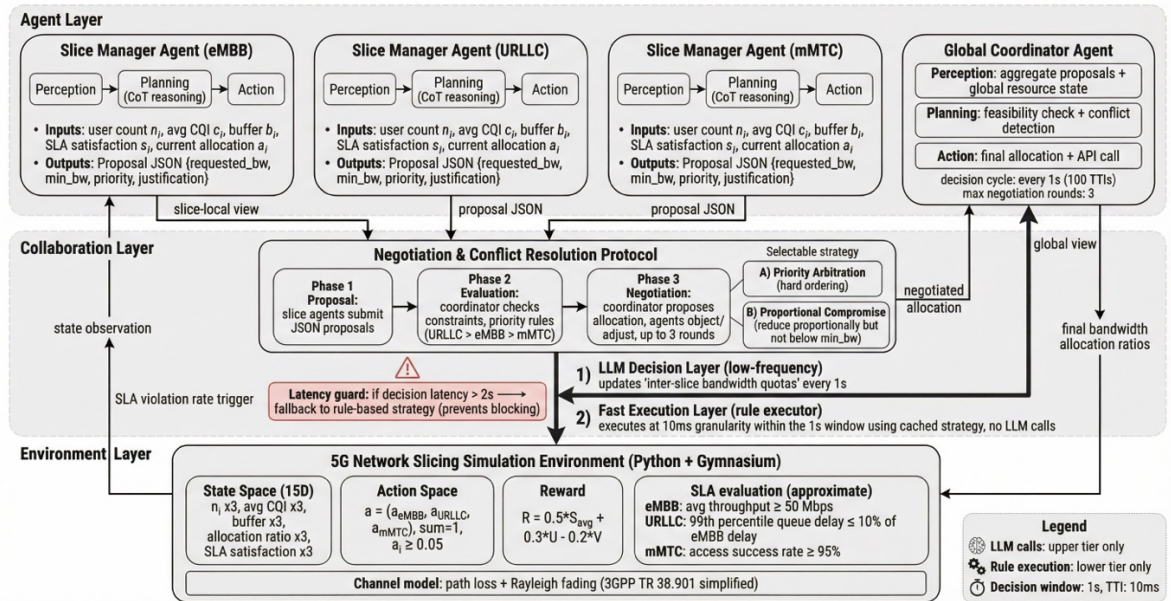


Figure 1. System Architecture

4.1 Overall Architecture and Two-Tier Scheduling

The framework comprises three layers, as shown in **Figure 1**: the **Agent Layer** defines specialised agents and their configurations; the **Collaboration Layer** implements inter-agent negotiation and conflict resolution; the **Environment Layer** provides a standardised simulation environment interface. The entire system is orchestrated via LangGraph, with GPT-4 API as the LLM backbone.

Two-Tier Scheduling Architecture (Core Engineering Design): To address the order-of-magnitude gap between LLM inference latency (hundreds of milliseconds to seconds) and network millisecond-level real-time requirements, this research adopts an explicit two-tier scheduling design:

- **Upper tier (LLM decision layer):** Triggers a multi-agent LLM decision every 1 second (100 TTIs) to update inter-slice bandwidth allocation quotas. Emergency decisions can be triggered early when SLA violation rates exceed a threshold.
- **Lower tier (fast execution layer):** Within each decision window, executes rule-based cached strategies at 10ms granularity (maintaining the upper-tier allocation), with no LLM inference involved.
- **Maximum decision latency threshold:** Set at 2 seconds; if exceeded, the system automatically falls back to rule-based strategies to prevent LLM inference from blocking system operation.

4.2 Agent Design

Each agent contains three core modules — Perception, Planning, and Action:

Slice Manager Agent: One per slice type, three in total. The Perception module converts the slice's numerical state (user count, average CQI, buffer occupancy, current SLA satisfaction rate) into structured natural language descriptions. The Planning module uses Chain-of-Thought (CoT) reasoning [18] to evaluate current resource demands. The Action module outputs resource demand proposals (requested bandwidth, minimum acceptable amount, priority declaration, justification).

Global Coordinator Agent: Only one, responsible for cross-slice coordination. The Perception module aggregates all slice agents' proposals and the global resource state. The Planning module determines whether slice proposals are compatible (whether total demand exceeds total resources); if compatible, proposals are directly approved; if in conflict, the negotiation protocol is triggered. The Action module executes the final bandwidth allocation scheme and calls the simulation environment API.

4.3 Negotiation Protocol Design

When the Global Coordinator Agent detects that slice proposals conflict (total resource demand exceeds available resources), the three-phase negotiation protocol is triggered:

Phase 1 (Proposal): Each Slice Manager Agent independently evaluates demand and submits a resource proposal in structured JSON format.

Phase 2 (Evaluation): The Global Coordinator Agent evaluates global constraint feasibility — checking whether total demand is excessive and whether hard priority rules exist (e.g., URLLC takes precedence over eMBB).

Phase 3 (Negotiation): If conflicts exist, the Global Coordinator Agent proposes an initial allocation scheme; affected Slice Manager Agents may raise objections and alternative proposals, with a **maximum of 3 rounds** to control latency and token costs. Consensus is reached through priority arbitration or proportional compromise.

The experiments will compare two strategies: **Priority Arbitration** (hard ordering: URLLC > eMBB > mMTC) and **Proportional Compromise** (each slice proportionally reduces demand, but not below the minimum acceptable amount).

4.4 Simulation Environment

A custom 5G network slicing simulation environment is built using Python and Gymnasium [20].

State Space (15 dimensions), defined and normalised as in **Table 3**:

Table 3. State space definition and normalisation in the Gymnasium-based environment.

Dimension	Meaning	Range	Normalisation
$n_i(\times 3)$	Current user count per slice	$[0, N_max]$	$/ N_max$
$\bar{c}_i(\times 3)$	Average CQI per slice	$[1, 15]$	$/ 15$
$b_i(\times 3)$	Buffer occupancy per slice	$[0, 1]$	Raw value
$a_i(\times 3)$	Current bandwidth allocation ratio per slice	$[0, 1]$	Raw value
$s_i(\times 3)$	SLA satisfaction rate per slice (sliding window)	$[0, 1]$	Raw value

Action Space: Continuous vector $a = (a_{eMBB}, a_{URLLC}, a_{mMTC})$, subject to $\sum a_i = 1$ and $a_i \geq 0.05$.

Reward Function: $R = w_S \cdot \bar{S} + w_U \cdot U - w_V \cdot V$, where $\bar{S}$, U , and V are normalised to $[0,1]$, representing the weighted average SLA satisfaction rate, bandwidth utilisation, and SLA violation penalty, respectively. The coefficients w_S, w_U, w_V are tunable hyperparameters satisfying $w_S + w_U + w_V = 1$.

SLA Evaluation (Simulation-Layer Approximate Definitions): The slice-wise SLA conditions used for evaluation are summarised in **Table 4**, and these metrics represent relative priority isolation rather than strict 3GPP end-to-end physical layer metrics.

Table 4. Simulation-layer SLA approximations and design intent per slice type.

Slice Type	SLA Condition	Design Intent	Typical Application
eMBB	Average user throughput ≥ 50 Mbps	High throughput guarantee	Video streaming
URLLC	99th percentile queuing delay $\leq 10\%$ of eMBB's delay	Relative high-priority guarantee	Industrial control
mMTC	Access success rate $\geq 95\%$	Massive access guarantee	IoT sensors

5. Baseline Method Implementation

The following baseline algorithms have been implemented and initially debugged:

Rule-based strategies: Fixed-ratio allocation (eMBB:URLLC:mMTC = 5:3:2), following commonly adopted traffic composition settings in network slicing simulations [23], and threshold-based dynamic adjustment triggered by SLA violation rates. The former serves as a performance lower bound, while the latter represents common engineering heuristics.

Deep Reinforcement Learning: Proximal Policy Optimisation (PPO) [22] implemented using Stable-Baselines3 with Multi-Layer Perceptron (MLP) policy networks, trained to convergence (early stopping when the average reward change rate over the last 50 episodes $< 1\%$, with a maximum of 1,000 episodes). Training convergence curves are recorded.

Single-Agent LLM: A single GPT-4 agent receives the global network state and outputs resource allocation decisions via CoT reasoning. No multi-agent collaboration. This baseline serves as a direct comparison against the multi-agent framework.

Experimental Design

Experimental Setup

Experimental Setup is fixed across all compared methods, and the full parameter settings are reported in **Table 5**.

Table 5. Experimental configuration for the two-tier LLM multi-agent framework.

Parameter	Setting
System bandwidth	100 MHz (reference NR n78 band)
Slice types	eMBB / URLLC / mMTC
User scale	100 users (standard scenario)
LLM decision cycle	Multi-agent LLM decision triggered every 1 second (100 TTIs)
Lower-tier execution granularity	10ms (TTI-level), rule-based cached strategy within decision windows
Maximum decision latency threshold	2 seconds (automatic fallback to rule-based strategy if exceeded)
Total simulation duration	500 LLM decision cycles per episode

Comparison Methods

Comparison Methods (M1–M5) are summarised in **Table 6**.

Table 6. Summary of compared methods and baselines (M1–M5).

ID	Method	Category	Description
M1	Fixed-Ratio	Rule-based	Fixed ratio allocation (5:3:2)
M2	Threshold-based	Rule-based	Dynamic adjustment based on SLA violation threshold
M3	PPO	Deep RL	MLP policy network, trained to convergence
M4	Single-Agent LLM	Single-agent LLM	GPT-4 + CoT, no collaboration
M5	Multi-Agent LLM	Multi-agent LLM (this work)	Slice agents + coordinator agent + negotiation

Evaluation Metrics

Evaluation Metrics and their computation rules are defined in **Table 7**.

Table 7. Evaluation metrics and computation definitions.

Metric	Computation
Per-slice SLA satisfaction rate	Proportion of decision cycles meeting SLA conditions
Weighted total throughput	Sum of all user throughputs
Bandwidth utilisation	Actual used bandwidth / Total allocated bandwidth
Inter-slice fairness	Jain's fairness index (based on per-slice SLA satisfaction rates)
Per-decision latency	End-to-end time from state input to decision output
API call cost	Total token consumption per episode (input + output)

Experiment 1: Core Performance Comparison (RQ1)

Purpose: Validate the performance differences of the multi-agent LLM framework relative to rule-based strategies, DRL, and single-agent LLM.

Method: Run M1–M5 for both the steady-state scenario (100 users) and the burst scenario (eMBB users doubled at cycle 250). Each setting is executed **30 independent runs with distinct random seeds**; LLM temperature is fixed to 0 to limit variability to environment and DRL exploration.

Core Deliverables: 1 main table (SLA satisfaction rate, throughput, utilisation, fairness, decision latency) + 1 trend curve showing performance under load variation.

Hypotheses and Contingency Analysis: Hypothesis: M5 (multi-agent) outperforms M4 (single-agent) and M3 (PPO). If the hypothesis is not supported, the analysis will examine whether multi-agent communication overhead offsets collaborative benefits, and whether the scenario complexity is insufficient to demonstrate multi-agent advantages.

Experiment 2: Negotiation Mechanism Validation (RQ2)

Purpose: Validate whether the inter-agent negotiation protocol provides structural advantages.

Method: Compare **M5 (full negotiation)** and **M5-no-neg (direct allocation)** under both scenarios, **30 seeds per setting** with the same randomness controls as above.

Core Deliverables: 1 comparison table + 1 inter-slice fairness (Jain index) comparison chart.

Hypotheses and Contingency Analysis: Hypothesis: the negotiation mechanism significantly improves fairness and SLA satisfaction rate under slice conflict scenarios. If not supported, the discussion will address whether centralised dictatorial allocation is already sufficient for simple scenarios, and whether the value of negotiation requires more complex multi-slice conflicts to manifest.

References

- [1] J. Tong *et al.*, “WirelessAgent: Large Language Model Agents for Intelligent Wireless Networks,” *arXiv.org*, 2025. <https://arxiv.org/abs/2505.01074> (accessed Mar. 02, 2026).
- [2] J. Shao *et al.*, “WirelessLLM: Empowering Large Language Models Towards Wireless Intelligence,” *Journal of Communications and Information Networks*, vol. 9, no. 2, pp. 99–112, Jun. 2024, doi: <https://doi.org/10.23919/jcin.2024.10582827>.
- [3] F. Jiang *et al.*, “Large Language Model Enhanced Multi-Agent Systems for 6G Communications,” *IEEE Wireless Communications*, vol. 31, no. 6, pp. 48–55, Aug. 2024, doi: <https://doi.org/10.1109/mwc.016.2300600>.
- [4] D. Wu *et al.*, “NetLLM: Adapting Large Language Models for Networking,” Aug. 2024, doi: <https://doi.org/10.1145/3651890.3672268>.
- [5] Y. Chen *et al.*, “NetGPT: An AI-Native Network Architecture for Provisioning Beyond Personalized Generative Services,” *IEEE Network*, pp. 1–1, Jan. 2024, doi: <https://doi.org/10.1109/mnet.2024.3376419>.
- [6] Z. Qu, W. Wang, Z. Yu, B. Sun, Y. Li, and X. Zhang, “LLM Enabled Multi-Agent System for 6G Networks: Framework and Method of Dual-Loop Edge-Terminal Collaboration,” *arXiv.org*, 2025. <https://arxiv.org/abs/2509.04993> (accessed Mar. 02, 2026).
- [7] H. Chergui, M. C. Cid, K. P. Sayyad, D. C. Mur, and C. Verikoukis, “Toward an Unbiased Collective Memory for Efficient LLM-Based Agentic 6G Cross-Domain Management,” *arXiv.org*, 2025. <https://arxiv.org/abs/2509.26200> (accessed Mar. 02, 2026).
- [8] F. Lotfi, H. Rajoli, and F. Afghah, “ORAN-GUIDE: RAG-Driven Prompt Learning for LLM-Augmented Reinforcement Learning in O-RAN Network Slicing,” *arXiv.org*, 2025. <https://arxiv.org/abs/2506.00576> (accessed Mar. 02, 2026).
- [9] S. Hong *et al.*, “MetaGPT: Meta Programming for Multi-Agent Collaborative Framework,” *arXiv.org*, Aug. 07, 2023. <https://arxiv.org/abs/2308.00352>
- [10] Q. Wu *et al.*, “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” *arXiv.org*, Oct. 03, 2023. <https://arxiv.org/abs/2308.08155>
- [11] G. Li, A. Kader, H. Itani, D. Khizbullin, and B. Ghanem, “CAMEL: Communicative Agents for ‘Mind’ Exploration of Large Language Model Society,” *arXiv.org*, 2023. <https://arxiv.org/abs/2303.17760>
- [12] C. Qian *et al.*, “ChatDev: Communicative Agents for Software Development,” *arXiv.org*, Jul. 18, 2023. <https://arxiv.org/abs/2307.07924>
- [13] I. Afolabi, T. Taleb, K. Samdanis, A. Ksentini, and H. Flinck, “Network Slicing and Softwarization: A Survey on Principles, Enabling Technologies, and Solutions,” *IEEE*

Communications Surveys & Tutorials, vol. 20, no. 3, pp. 2429–2453, 2018, doi: <https://doi.org/10.1109/comst.2018.2815638>.

[14] R. Li *et al.*, “Deep Reinforcement Learning for Resource Management in Network Slicing,” *IEEE Access*, vol. 6, pp. 74429–74441, 2018, doi: <https://doi.org/10.1109/access.2018.2881964>.

[15] V. Sciancalepore, X. Costa-Perez, and A. Banchs, “RL-NSB: Reinforcement Learning-Based 5G Network Slice Broker,” *IEEE/ACM Transactions on Networking*, vol. 27, no. 4, pp. 1543–1557, Aug. 2019, doi: <https://doi.org/10.1109/tnet.2019.2924471>.

[16] L. Wang *et al.*, “A Survey on Large Language Model based Autonomous Agents,” *arXiv.org*, Sep. 07, 2023. <https://arxiv.org/abs/2308.11432v2>

[17] H. Zhou *et al.*, “Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities,” *IEEE Communications Surveys & Tutorials*, pp. 1–1, Jan. 2024, doi: <https://doi.org/10.1109/comst.2024.3465447>.

[18] J. Wei *et al.*, “Chain of Thought Prompting Elicits Reasoning in Large Language Models,” *arXiv:2201.11903 [cs]*, Oct. 2022, Available: <https://arxiv.org/abs/2201.11903>

[19] P. Caballero, A. Banchs, G. De Veciana, and X. Costa-Perez, “Network Slicing Games: Enabling Customization in Multi-Tenant Mobile Networks,” *IEEE/ACM Transactions on Networking*, vol. 27, no. 2, pp. 662–675, Apr. 2019, doi: <https://doi.org/10.1109/tnet.2019.2895378>.

[20] G. Brockman *et al.*, “OpenAI Gym,” *arXiv.org*, 2016. <https://arxiv.org/abs/1606.01540>

[21] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Neural Information Processing Systems*, 2020. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>

[22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimisation Algorithms,” *arXiv.org*, Aug. 28, 2017. <https://arxiv.org/abs/1707.06347>

[23] A. R. Nasser and O. Y. Alani, “Investigation of Multiple Hybrid Deep Learning Models for Accurate and Optimized Network Slicing,” *Computers*, vol. 14, no. 5, p. 174, May 2025, doi: <https://doi.org/10.3390/computers14050174>.

尚需完成的任务 Work to do:

1. **Complete Gymnasium simulation environment:** Implement the 15-dimensional state space, action space, reward function, and SLA evaluation logic. Ensure rule-based baseline (M1) runs correctly and outputs all metrics.
2. **Minimal dual-agent version:** Implement 1 slice agent + 1 coordinator agent as a minimal end-to-end pipeline via LangGraph, completing the full "perceive → propose → evaluate → allocate → execute" flow.
3. **First round of baseline comparison data:** Run at least M1 (rules), M3 (PPO), and M4 (single-agent LLM) comparisons to produce 1 core metrics table.
4. **Full multi-agent framework:** Extend to 3 slice agents + 1 coordinator agent with the negotiation protocol implemented.
5. **Complete execution of Experiments 1 and 2:** Finish all core comparisons and negotiation mechanism validation.
6. **Report writing:** Complete all chapters, with emphasis on figure/table quality for experimental results and honest Discussion.

存在问题 Problems:

1. **LLM decision latency vs. network real-time requirements:** LLM inference latency (hundreds of milliseconds to seconds) is orders of magnitude greater than network millisecond-level real-time requirements. Multi-agent scenarios further amplify latency through multiple inter-agent calls. The two-tier scheduling architecture (§4.1) has been adopted in response, but its practical effectiveness remains to be validated.

2. **API call costs:** GPT-4 requires multiple API calls per decision in multi-agent negotiation scenarios, with significant token consumption and costs per episode. Total costs must be estimated early, and a budget cap established.
3. **Evaluation data authenticity:** The simulation environment is based on synthetic traffic models with queuing theory approximations for SLA evaluation, which may not fully represent real network scenarios.
4. **Schedule risk:** The core multi-agent framework is not yet fully implemented; the remaining period requires simultaneous system implementation, experiments, and report writing. This has been mitigated through scope convergence (2 RQs, 2 core experiments).

拟采取的办法 **Solutions:**

1. **Two-tier architecture validation:** The experiments will compare "pure rule-based strategy performance within the 1-second window" against "performance improvement after LLM updates" to quantify the marginal value of LLM decisions, with a 2-second maximum decision latency threshold as a deployment feasibility criterion.
2. **Cost control:** Estimate per-episode token consumption limits from trial runs before experiments; set a total budget. The negotiation round hard limit of 3 rounds controls token costs at the mechanism level.
3. **Honest discussion of limitations:** The Discussion chapter will explicitly list Threats to Validity — simulation approximation boundaries, relative SLA definition limitations, and synthetic data representativeness.

论文结构 **Structure of the final report: (Chapter headings and section sub headings)**

Abstract

Keywords

Chapter 1: Introduction

- 1.1 Background and Motivation
- 1.2 Problem Statement
- 1.3 Research Objectives
- 1.4 Scope and Assumptions
- 1.5 Contributions
- 1.6 Report Organisation

Chapter 2: Background

- 2.1 Network Slicing Architecture and Resource Management Models
- 2.2 SLA-Aware Scheduling and Two-Tier Control Mechanisms
- 2.3 LLM-Based Agent Frameworks in Network Management
- 2.4 Multi-Agent Coordination and Negotiation Mechanisms
- 2.5 Identified Research Gaps

Chapter 3: Design and Implementation

- 3.1 System Overview and Architectural Principles
- 3.2 Two-Tier Scheduling Framework
- 3.3 Agent Role Definition and Internal Logic
- 3.4 Proposal–Evaluation–Negotiation Protocol
- 3.5 Gymnasium-Based Simulation Environment
- 3.6 LangGraph-Orchestrated Agent Workflow
- 3.7 Baseline Methods and Implementation Details

Chapter 4: Results and Discussion

- 4.1 Experimental Design and Evaluation Metrics
- 4.2 Core Performance Comparison
- 4.3 Negotiation Mechanism Validation
- 4.4 Result Analysis
- 4.5 Threats to Validity and Limitations

4.6 Practical Implications

Chapter 5: Conclusion and Further Work

5.1 Conclusion

5.2 Reflection

5.3 Further Work

References

Appendices

Disclaimer

Acknowledgement of using Generative AI

Project specification

Early-term progress report

Mid-term progress report

Supervision log

Prompt Templates for Each Agent Role

Agent Message Format (JSON Schema)

Simulation Environment Parameters

DRL Training Convergence Curves

Risk and Environmental Impact Assessment